CHAPTER 7

NEURAL NETWORKS

7.1 Introduction

Neural networks are a foundational component of modern machine learning and artificial intelligence. Inspired by the structure of the human brain, a neural network consists of layers of interconnected units called neurons that process data in stages. These models are particularly effective in tasks like image classification, natural language processing, and regression analysis.

7.2 Perceptron: The Basic Unit

The perceptron is the simplest type of artificial neuron, introduced by Frank Rosenblatt in 1958. It forms the building block of a neural network and functions similarly to linear classifiers that use hard thresholding, as you may recall from the previous chapter.

The linear equation for the weighted sum is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b, \quad y = f(z) \tag{7.1}$$

Here:

- x_i are the input features
- w_i are the corresponding weights
- b is a bias term
- f is an activation function, a step function in this case.

$$y = f(z) \text{ (step)} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases} = 0 \tag{7.2}$$

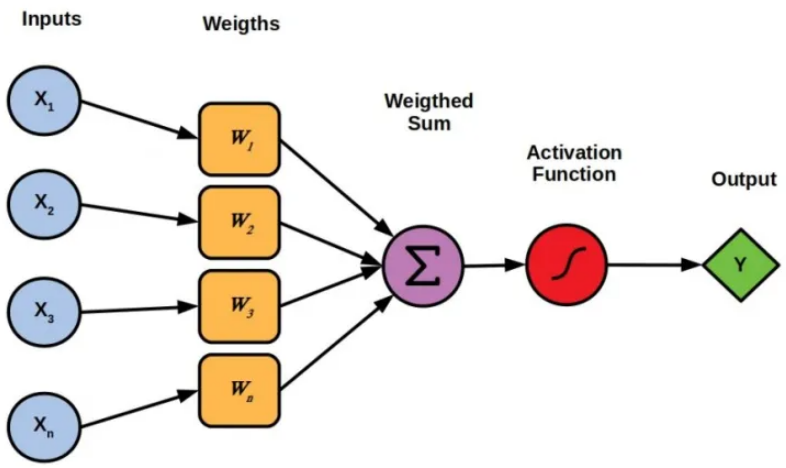


Figure 7.1: A neural network with single neuron perceptron

Image credit: https://www.linkedin.com/pulse/perceptron-the-basic-building-block-neural-networks-ayush-meharkure-p46if/

A perceptron takes several input values, applies weights to them, sums them together with a bias term, and then passes the result through an **activation function** — in our case, a step function in the original model. It can be used to separate linearly separable data.

Example: Let $x = [2, 3]$, $w = [0.4, -0.7]$, $b = 0.2$:

$$\begin{aligned} z &= (0.4) \cdot 2 + (-0.7) \cdot 3 + 0.2 \\ &= -1.1 \\ y = f(z) &= 0 \text{ using step function.} \end{aligned}$$

Although simple, the perceptron cannot handle tasks that are not linearly separable (such as the XOR function). To address this limitation, we combine multiple perceptrons in layers to form more powerful models known as multi-layer neural networks.

7.3 Network Layers

7.3.1 Input Layer

The input layer is the first layer in a neural network. It serves as the interface between raw data and the computational structure of the network. Each neuron in this layer represents one feature or attribute of the input data. For instance, in image data, each pixel could be one input node. This layer does not perform any computations beyond passing the inputs to the next layer.

7.3.2 Hidden Layers

Hidden layers are intermediate layers that lie between the input and output layers. These layers are called "hidden" because their outputs are not part of the final network output but are used internally.

Weight Matrix: The full set of weights between two layers can be written as a matrix $W^{(l)}$, where each row corresponds to the weights leading into a single neuron in the current layer. If the previous layer has n neurons and the current layer has m neurons, then $W^{(l)}$ **is an** $m \times n$ **matrix** and the bias vector will have m entries each associated with one neuron.

$$W^{(l)} = \begin{bmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \cdots & w_{1n}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \cdots & w_{2n}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1}^{(l)} & w_{m2}^{(l)} & \cdots & w_{mn}^{(l)} \end{bmatrix}_{m \times n} \quad ; \quad b^{(l)} = \begin{bmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_m^{(l)} \end{bmatrix}_{m \times 1}$$

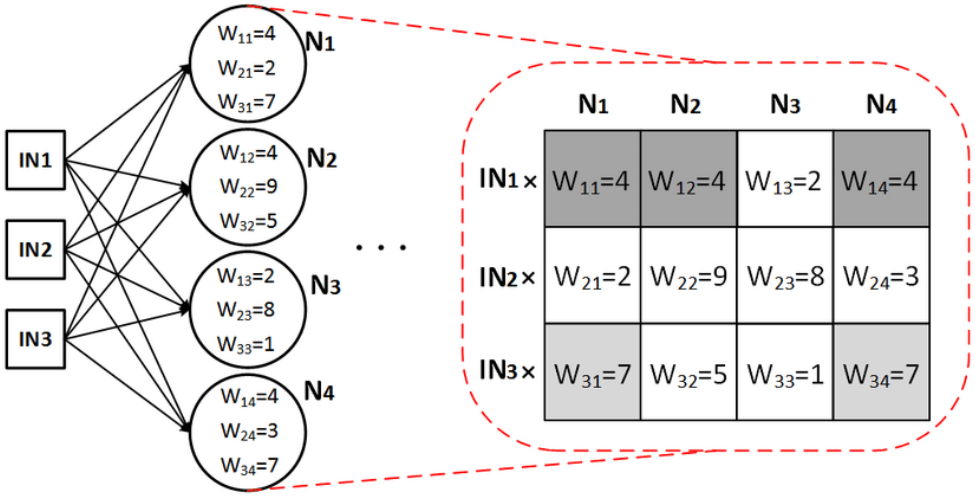


Figure 7.2: Weight Matrix for a hidden layer

Image credit: https://medium.com/coinmonks/the-mathematics-of-neural-network-60a112dd3e05

Each neuron in a hidden layer performs the following computation:

$$z_j^{(l)} = \sum_{i=1}^{n} w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = f(z_j^{(l)}) \tag{7.3}$$

Where:

- $a_i^{(l-1)}$ is the activation from neuron i in the previous layer
- $w_{ji}^{(l)}$ is the weight from neuron i in layer $l-1$ to neuron j in layer l

- $b_j^{(l)}$ is the bias for neuron j in layer l
- f is the activation function applied element-wise

In a vector representation:

$$z^{(l)} = \underbrace{\begin{bmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \cdots & w_{1n}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \cdots & w_{2n}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1}^{(l)} & w_{m2}^{(l)} & \cdots & w_{mn}^{(l)} \end{bmatrix}}_{W^{(l)} \in \mathbb{R}^{m \times n}} \cdot \underbrace{\begin{bmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ \vdots \\ a_n^{(l-1)} \end{bmatrix}}_{a^{(l-1)} \in \mathbb{R}^{n \times 1}} + \underbrace{\begin{bmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_m^{(l)} \end{bmatrix}}_{b^{(l)} \in \mathbb{R}^{m \times 1}} \in \mathbb{R}^{m \times 1} \tag{7.4}$$

more consisely,

$$z^{(l)} = W^{(l)} \cdot a^{(l)} + b^{(l)} \tag{7.5}$$

- $W^{(l)}$: Weight matrix of shape $m \times n$, where each row corresponds to a neuron in the current layer, and each column to a neuron in the previous layer.
- $a^{(l-1)}$: Activation vector from the previous layer, of shape $n \times 1$. For the first layer the activation vector is the original input vector, X with n attributes.
- $b^{(l)}$: Bias vector for the current layer, with one bias per neuron, of shape $m \times 1$.
- $z^{(l)}$: Pre-activation vector of the current layer, of shape $m \times 1$, computed as the weighted sum of previous activations plus bias.

7.3.3 Activation Function

An activation function is a non-linear function applied to the output of each neuron after the weighted sum and bias addition.

$$a^{(l)} = f(z^{(l)}) = \begin{bmatrix} f(z_1^{(l)}) \\ f(z_2^{(l)}) \\ \vdots \\ f(z_m^{(l)}) \end{bmatrix} \in \mathbb{R}^{m \times 1} \tag{7.6}$$

- f: The activation function (e.g., sigmoid, ReLU, tanh), applied element-wise.
- $z^{(l)}$: The pre-activation vector computed from the weighted sum and bias.
- $a^{(l)}$: The output activations from the current layer, used as input to the next layer (or final prediction if it's the output layer).

It introduces non-linearity into the model, allowing it to learn complex patterns. Common activation functions include:

- **Step**: $\text{step}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$

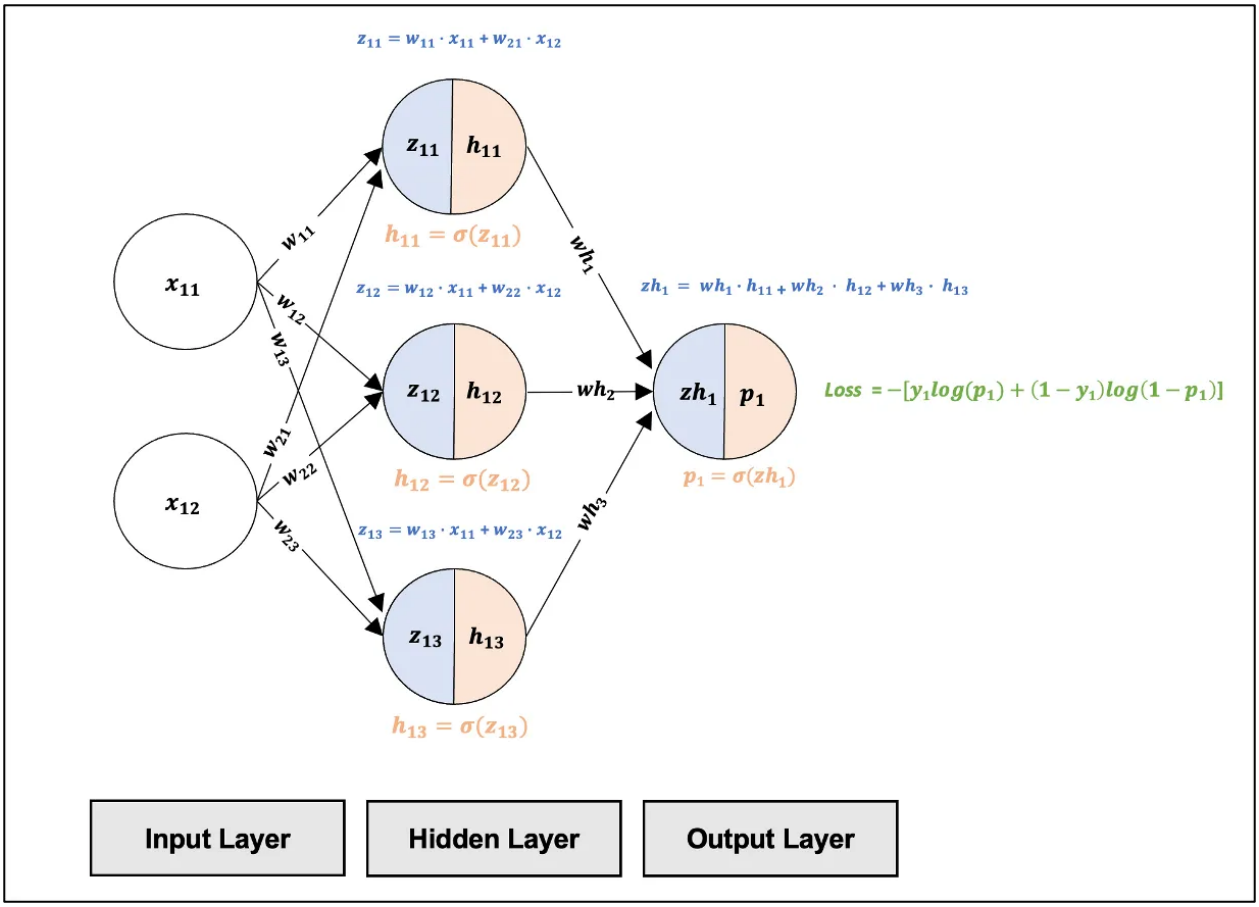


Figure 7.3: Hidden Layer of Neural Network

Image credit: https://pub.towardsai.net/the-multilayer-perceptron-built-and-implemented-from-scratch-70d6b30f1964

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$ maps input to (0,1)
- **Tanh**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$.
- ReLU: $\mathbf{ReLU}(z) = \mathbf{max}(0, z)$.

Each has different properties affecting training dynamics and convergence.

7.3.4 Output Layer

The output layer produces the final result of the neural network. The type of activation function used in this layer depends on the task:

- **Sigmoid**: Used for binary classification problems, maps output to a probability in (0,1)
- **Softmax**: Used for multi-class classification, gives a probability distribution over classes
- **Identity**: Used in regression tasks, outputs real-valued predictions

7.4 Understanding the Weight Matrix and Activation Function

The weight matrix $W^{(l)}$ plays a crucial role in how information flows and transforms across layers of a neural network. It is responsible for linearly combining the activations from the previous layer and shaping how those activations influence the neurons in the current layer.

For a given layer l, the weight matrix $W^{(l)}$ has dimensions $m \times n$, where:

- n is the number of neurons in the previous layer $l-1$
- m is the number of neurons in the current layer l

The output of the linear transformation is:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \tag{7.7}$$

This vector $z^{(l)}$ is then passed through an activation function f to introduce non-linearity:

$$a^{(l)} = f(z^{(l)}) \tag{7.8}$$

Role of the Weight Matrix

- Each row in $W^{(l)}$ contains the weights associated with one neuron in the current layer.
- Each column in $W^{(l)}$ corresponds to the contribution from one neuron in the previous layer.
- The dot product $W^{(l)} a^{(l-1)}$ produces a vector of weighted sums, one per neuron in layer l.

Relation to Activation Functions

The activation function f operates element-wise on the result of the matrix multiplication $z^{(l)}$. Without f, the entire network would be a stack of linear transformations—essentially equivalent to a single matrix multiplication. The use of f makes it possible for the network to model complex non-linear relationships in data.

For example, in the step function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases} \tag{7.9}$$

The activation function thresholds the result of each neuron's computation, allowing the network to model logical decision boundaries like AND, OR, or XOR (with enough layers).

Thus, the weight matrix defines how signals propagate and interact, and the activation function defines how these signals are interpreted at each neuron.

Detailed Example: Two-Layer Feedforward Network

Input:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Weights and Biases:

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.6 \\ 0.3 & 0.8 \end{bmatrix}, \qquad b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$
$$W^{(2)} = \begin{bmatrix} 1.2 & -0.5 \end{bmatrix}, \qquad b^{(2)} = 0.3$$

Step 1: First Linear Transformation (Hidden Layer)

$$\begin{aligned} z^{(1)} &= W^{(1)}x + b^{(1)} \\ &= \begin{bmatrix} 0.5 \cdot 1 + (-0.6) \cdot 2 + 0.1 \\ 0.3 \cdot 1 + 0.8 \cdot 2 - 0.2 \end{bmatrix} \\ &= \begin{bmatrix} -0.6 \\ 1.7 \end{bmatrix} \end{aligned}$$

Step 2: Activation Function (Step Function)

$$a^{(1)} = f(z^{(1)}) = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 0 \\ 1.7 \end{bmatrix}$$

Step 3: Second Linear Transformation (Output Layer)

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = 1.2 \cdot 0 + (-0.5) \cdot 1 + 0.3 = -0.2$$

Step 4: Final Activation (Step Function)

$$\hat{y} = f(z^{(2)}) = \text{sigmoid}(-0.2) = \frac{1}{1 + e^{-(-0.2)}}$$

In this example:

- The first layer transformed the input using weights and biases.
- The step activation introduced non-linearity, turning negative values to 0 and non-negatives to 1.
- The second layer used the resulting activations to compute a final output.

7.5 Feedforward Neural Networks

A feedforward neural network (FNN) is the most basic type of artificial neural network where connections between the nodes do not form cycles. The network processes input data layer-by-layer, from input to output, applying transformations at each stage.

Feedforward networks have layers where data flows one-way:

$$a^{(1)} = f^{(1)}(W^{(1)}x + b^{(1)})$$
$$a^{(2)} = f^{(2)}(W^{(2)}a^{(1)} + b^{(2)}) = \hat{y}$$

Each activation $a^{(l)}$ is passed to the next layer until the final output is produced.

General Structure

Assume a network with L layers (excluding the input layer), where each layer l has $n^{(l)}$ neurons.

Given input $x \in \mathbb{R}^{n^{(0)}}$ (e.g., a feature vector), the feedforward steps are as follows:

Step 1: Weighted Sum at Each Neuron

Each neuron computes a weighted sum of its inputs plus a bias:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \tag{7.10}$$

Here:

- $W^{(l)} \in \mathbb{R}^{n^{(l)} \times n^{(l-1)}}$ is the weight matrix of layer l
- $a^{(l-1)} \in \mathbb{R}^{n^{(l-1)}}$ is the activation output from the previous layer (or input x if $l = 1$)
- $b^{(l)} \in \mathbb{R}^{n^{(l)}}$ is the bias vector of layer l

Step 2: Activation Function

Each $z^{(l)}$ is passed through a non-linear activation function σ to produce activations for the current layer:

$$a^{(l)} = \sigma(z^{(l)}) \tag{7.11}$$

Common choices for σ include:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- ReLU: $\sigma(z) = \max(0, z)$
- Tanh: $\sigma(z) = \tanh(z)$

Final Output Layer

The output layer's activation, $a^{(L)}$, gives the network's prediction:

$$\hat{y} = a^{(L)} = \sigma(z^{(L)}) \tag{7.12}$$

Example 1: Single Layer Network (Result Prediction)

Consider a single-layer network (with sigmoid activation function) used to predict whether a student passes an exam based on two features:

- x_1: Number of study hours
- x_2: Number of hours of sleep

Input vector:

$$x = \begin{bmatrix} 5 \\ 7 \end{bmatrix}, \quad W = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}, \quad b = -3$$

Compute the linear combination:

$$z = Wx + b = 0.4 \cdot 5 + 0.6 \cdot 7 - 3 = 2 + 4.2 - 3 = 3.2$$

Apply the sigmoid activation function:

$$\hat{y} = \sigma(3.2) = \frac{1}{1 + e^{-3.2}} \approx 0.9608$$

The model predicts a 96% chance the student will pass based on study and sleep hours.

Example 2: Single-Layer Network (Spam Classifier)

Let us consider a single layer neural network with sigmoid activation function. Suppose we want to predict whether an email is spam based on two features:

- x_1: Number of links in the email

- x_2: Number of exclamation marks

Input vector:

$$x = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Let the weights and bias be:

$$w = \begin{bmatrix} 0.7 & 1.5 \end{bmatrix}, \quad b = -2$$

Step 1: Compute the weighted sum:

$$\begin{aligned} z &= 0.7 \cdot 3 + 1.5 \cdot 1 - 2 \\ &= 2.1 + 1.5 - 2 = 1.6 \end{aligned}$$

Step 2: Apply sigmoid function:

$$y = \sigma(z) = \frac{1}{1 + e^{-1.6}} \approx 0.832$$

The probability of spam is approximately 83.2%.

Example 3: 3-Layer Network (Loan Approval) — with 3 Neurons per Layer

Considering a 3-layer neural network where each hidden layer has 3 neurons for the previous example. We are using sigmoid activation function in every layer. We are still using three input features:

- x_1: Applicant's income (normalized)
- x_2: Credit score (normalized)
- x_3: Number of existing loans

Input vector:

$$x = \begin{bmatrix} 0.8 \\ 0.6 \\ 2 \end{bmatrix}$$

Layer 1 (Hidden Layer 1):

$$W^{(1)} = \begin{bmatrix} 0.2 & 0.4 & -0.5 \\ -0.3 & 0.1 & 0.6 \\ 0.5 & -0.2 & 0.3 \end{bmatrix}, \qquad b^{(1)} = \begin{bmatrix} 0.1 \\ 0.2 \\ -0.1 \end{bmatrix}$$

$$\textbf{Pre-activation vector, } z^{(1)} = W^{(1)}x + b^{(1)}$$

$$= \begin{bmatrix} 0.16 + 0.24 - 1.0 + 0.1 \\ -0.24 + 0.06 + 1.2 + 0.2 \\ 0.4 - 0.12 + 0.6 - 0.1 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 1.22 \\ 0.78 \end{bmatrix}$$

$$\textbf{Apply activation (sigmoid): } a^{(1)} = \sigma(z^{(1)}) = \begin{bmatrix} 0.3775 \\ 0.772 \\ 0.6859 \end{bmatrix}$$

Layer 2 (Hidden Layer 2):

$$W^{(2)} = \begin{bmatrix} 0.3 & 0.7 & -0.2 \\ -0.1 & 0.4 & 0.5 \\ 0.2 & -0.3 & 0.6 \end{bmatrix}, \qquad b^{(2)} = \begin{bmatrix} -0.2 \\ 0.3 \\ 0.1 \end{bmatrix}$$

$$\textbf{Pre-activation vector: } z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$= \begin{bmatrix} 0.1133 + 0.5404 - 0.1372 - 0.2 \\ -0.0378 + 0.3088 + 0.343 + 0.3 \\ 0.0755 - 0.2316 + 0.4115 + 0.1 \end{bmatrix} \approx \begin{bmatrix} 0.3165 \\ 0.914 \\ 0.354 \end{bmatrix}$$

$$\textbf{Apply activation: } a^{(2)} = \sigma(z^{(2)}) = \begin{bmatrix} 0.5785 \\ 0.7138 \\ 0.5875 \end{bmatrix}$$

Output Layer:

$$W^{(3)} = \begin{bmatrix} 1.0 & -1.5 & 0.6 \end{bmatrix}, b^{(3)} = 0.1$$

$$\textbf{Pre-activation vector, } z^{(3)} = 1.0 \cdot 0.5785 - 1.5 \cdot 0.7138 + 0.6 \cdot 0.5875 + 0.1$$

$$= -0.0397$$

$$\textbf{Apply sigmoid activation: } \hat{y} = \sigma(z^{(3)}) = \frac{1}{1 + e^{0.0397}} \approx 0.4901$$

The model predicts a 49.01% probability of loan approval.

7.6 Activation and Loss Functions

In this section, we define different activation function and the equation of Loss for them.

7.6.1 Sigmoid (Binary Classification)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\mathcal{L}_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

7.6.2 Softmax (Multiclass Classification)

$$\hat{y}_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

$$\mathcal{L}_{\text{CCE}} = -\sum_j y_j \log(\hat{y}_j)$$

7.6.3 Linear (Regression)

$$\hat{y} = W \cdot X + b$$

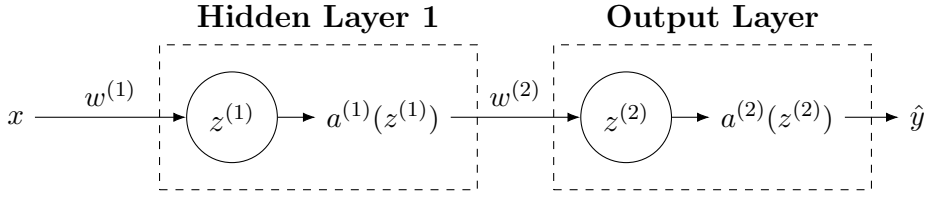
$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

7.7 Training: Gradient Descent and Backpropagation

7.7.1 Backpropagation in Feedforward Neural Networks

Backpropagation is the core algorithm used to train feedforward neural networks by updating the weights and biases to minimize a loss function. It computes the gradient of the loss function with respect to each parameter using the chain rule of calculus.

Assume a network with L layers, where each layer l has activations $a^{(l)}$, weights $W^{(l)}$, biases $b^{(l)}$, and pre-activations $z^{(l)}$.



For simplicity, let us consider $L = 2$, with activation functions, $a^{(1)}$ for the hidden layer and $a^{(2)}$ for the output layer. The weight matrix for the hidden layer is $w^{(1)}$ and for the output layer is $w^{(2)}$.

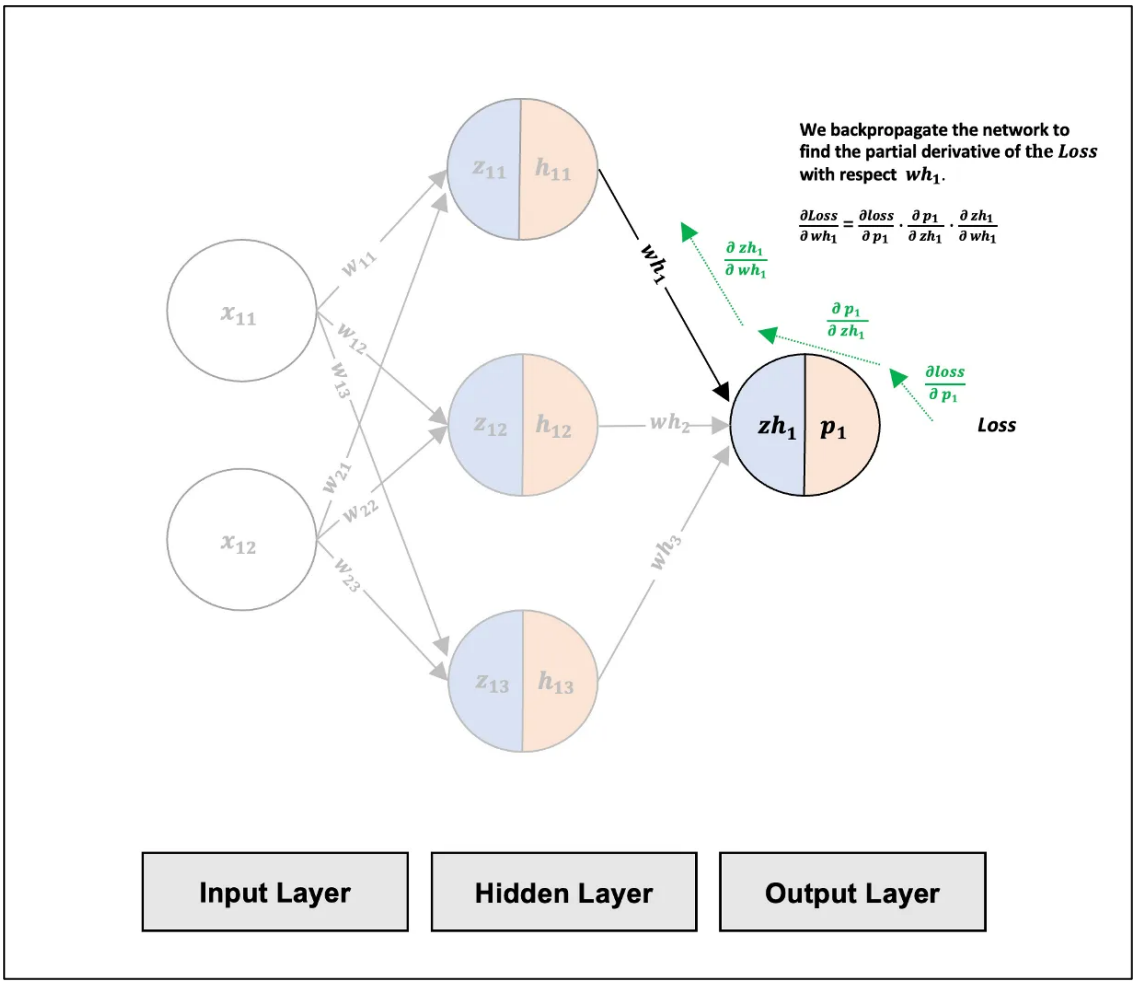


Figure 7.4: Image credit: https://pub.towardsai.net/the-multilayer-perceptron-built-and-implemented-from-scratch-70d6b30f1964

We have to update the weight vectors as

$$w^{(2)} = w^{(2)} + \alpha \frac{\partial \mathcal{L}(y)}{\partial w^{(2)}}$$
$$b^{(2)} = b^{(2)} + \frac{\partial \mathcal{L}}{\partial b^{(2)}}$$
$$w^{(2)} = w^{(2)} - \alpha \frac{\partial \mathcal{L}(y)}{\partial w^{(2)}}$$
$$b^{(2)} = b^{(2)} + \frac{\partial \mathcal{L}}{\partial b^{(2)}}$$

7.7.2 Computing the Gradient at the Output Layer

At the output layer L, the computation of $\frac{\partial \mathcal{L}}{\partial W^{(L)}}$ is similar to the gradient calculation used in the basic gradient descent algorithm.

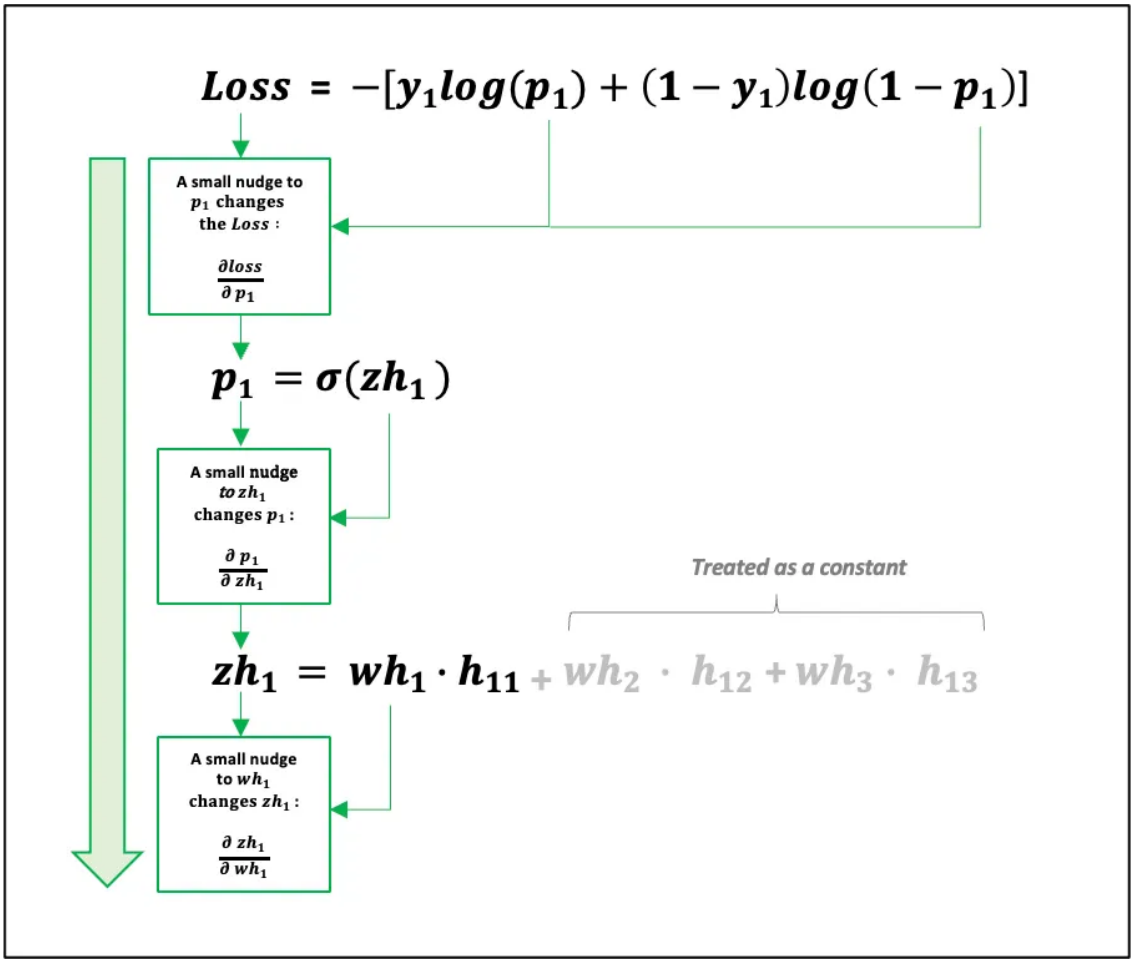


Figure 7.5: Image credit: https://pub.towardsai.net/the-multilayer-perceptron-built-and-implemented-from-scratch-70d6b30f1964

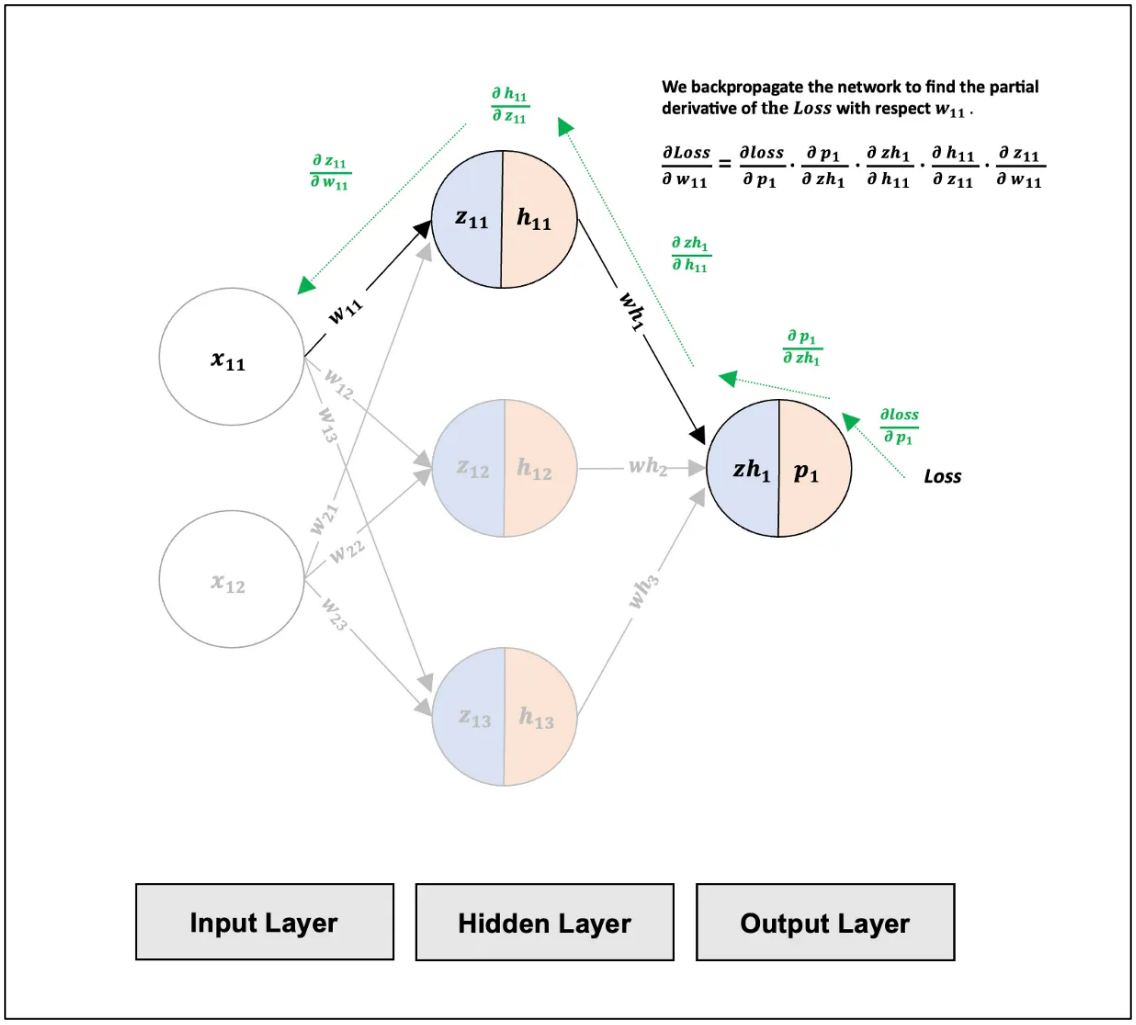


Figure 7.6: Image Credit: https://pub.towardsai.net/the-multilayer-perceptron-built-and-implemented-from-scratch-70d6b30f1964

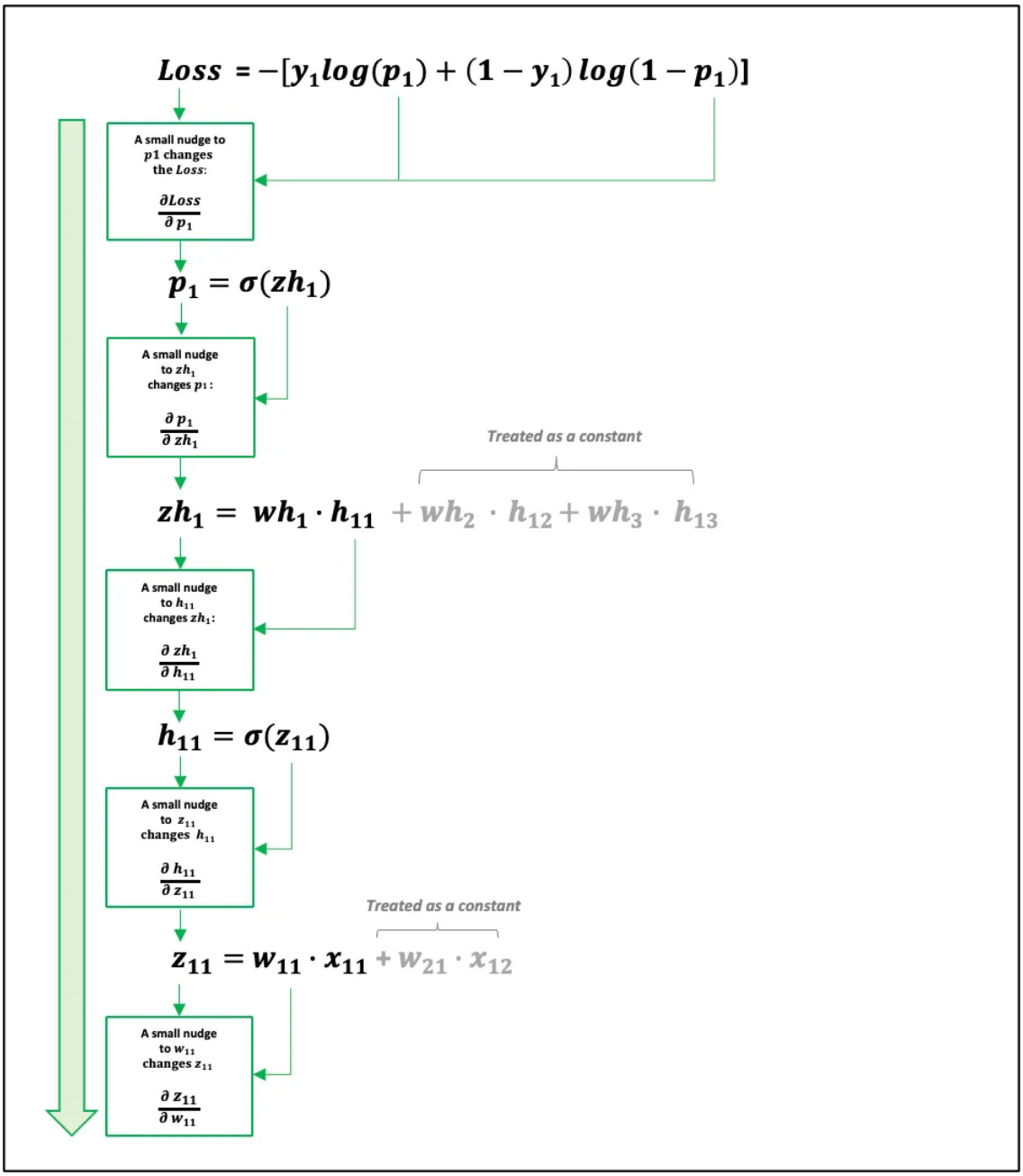


Figure 7.7: Image credit: https://pub.towardsai.net/the-multilayer-perceptron-built-and-implemented-from-scratch-70d6b30f1964

The output layer has a single neuron. The linear equation to this neuron is given by:

$$Z_1^{(L)} = W^{(L)} \cdot a^{(L-1)} + b^{(L)} = w_1^{(L)} a_1^{(L-1)} + w_2^{(L)} a_2^{(L-1)} + \cdots + w_p^{(L)} a_p^{(L-1)} + b^{(L)} \tag{7.13}$$

Here, p is the number of neurons in the previous layer (layer $L-1$), and each $a_i^{(L-1)}$ is the activation output from neuron i in that layer.

$$\frac{\partial \mathcal{L}}{\partial w_i^{(L)}} = \frac{\partial \mathcal{L}}{\partial a^L} \frac{\partial a^L}{\partial Z_1} \frac{\partial Z_1}{\partial w_i}, \text{ using chain rule. Recall, } \hat{y} = a^{(L)}(Z_1),$$

Recall that: $z_1^{(L)} = \sum_i w_{1i}^{(L))} a_i^{(1)} + b_j^{(2)}$, so,

$$\frac{\partial z_1^{(L)}}{\partial w_{1i}^{(L)}} = a_i^{(L-1)}.$$

Hence,

$$\frac{\partial \mathcal{L}}{\partial w_i^{(L)}} = \frac{\partial \mathcal{L}}{\partial a^L} \frac{\partial a^L}{\partial Z_1} a_i^{(L-1)}$$

and by using, $\frac{\partial Z_1}{\partial b_i^{(L)}} = 1$

$$\frac{\partial \mathcal{L}}{\partial b_i^{(L)}} = \frac{\partial \mathcal{L}}{\partial a^L} \frac{\partial a^L}{\partial Z_1} \cdot 1$$

Generally, this can be written as:

$$\frac{\partial \mathcal{L}}{\partial W^{(L)}} = \frac{\partial \mathcal{L}}{\partial a^L} \frac{\partial a^L}{\partial Z_1} \cdot a^{(L-1)}; \tag{7.14}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(L)}} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial Z_1}. \tag{7.15}$$

Finally, the update rules at the output layer, L can be written as:

$$W^{(L)} \leftarrow W^{(L)} - \alpha \frac{\partial \mathcal{L}}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial Z_1} \cdot (a^{(L-1)})^\top \tag{7.16}$$

$$b^{(L)} \leftarrow b^{(L)} - \alpha \frac{\partial \mathcal{L}}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial Z_1}. \tag{7.17}$$

Note: Here, The activation vector $a^{(L)}$ is transposed to turn it into a row vector so it can be multiplied with the error. This gives a full matrix of weight updates, matching the shape of $W^{(L)}$.

7.7.3 Understanding the Derivative of Activation with Respect to Pre-Activation

The derivative of the activation $a^{(i)}$ with respect to the input $Z^{(i)}$ at layer i is given by:

$$\frac{da^{(i)}}{dZ^{(i)}} = \phi'\left(Z^{(i)}\right) \tag{7.18}$$

Examples:

- **Sigmoid activation:**

$$\phi(z) = \frac{1}{1+e^{-z}} \quad \Rightarrow \quad \phi'(z) = \phi(z)(1-\phi(z))$$

So,

$$\frac{da^{(i)}}{dZ^{(i)}} = a^{(i)}(1-a^{(i)}) \tag{7.19}$$

- **ReLU activation:**

$$\phi(z) = \max(0,z) \quad \Rightarrow \quad \phi'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

So,

$$\frac{da^{(i)}}{dZ^{(i)}} = \begin{cases} 1 & \text{if } Z^{(i)} > 0 \\ 0 & \text{otherwise} \end{cases} \tag{7.20}$$

7.7.4 Understanding the Derivative of Loss with Respect to Output Activation

$\frac{\partial \mathcal{L}}{\partial a^{(L)}}$ represents the derivative of the loss function $\mathcal{L}$ with respect to the activation output of the final layer — that is, how the loss changes as the network's prediction changes. The exact form depends on the loss function used. Let $\hat{y} = a^{(L)}$ be the activation at the output layer and y be the true label.

Mean Squared Error (MSE) Loss:

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 \tag{7.21}$$

$$\frac{d\mathcal{L}}{da^{(L)}} = \hat{y} - y \tag{7.22}$$

Binary Cross-Entropy (BCE) Loss

$$\mathcal{L} = -\left(y \log \hat{y} + (1-y)\log(1-\hat{y})\right) \tag{7.23}$$

$$\frac{d\mathcal{L}}{da^{(L)}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right) \tag{7.24}$$

7.7.5 Defining Error, $\frac{d\mathcal{L}}{dZ^{(l)}}$

Let us define,

$$\frac{\partial \mathcal{L}}{\partial Z^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial Z^{(l)}} = D^{(l)} \tag{7.25}$$

where, l is the layer.

The term $\frac{d\mathcal{L}}{dZ^{(l)}} = D_i^{(l)}$ represents how the loss $\mathcal{L}$ changes with respect to the $Z_i^{(l)}$ of the i'th neuron at layer l.

The Error $D^{(l)}$ combines:

- The sensitivity of the loss to the neuron's activation: $\frac{\partial \mathcal{L}}{\partial a^l}$
- The sensitivity of the neuron's activation to its Z: $\frac{da}{dZ}$

7.7.6 Update Rule at the output layer

For the 2-layer feedforward neural network,

$$W^{(2)} \leftarrow W^{(2)} - \alpha D^{(2)} (a^{(L-1)})^\top \tag{7.26}$$

$$b^{(2)} \leftarrow b^{(2)} - \alpha D^{(2)} \tag{7.27}$$

7.7.7 Computing gradient at the Hidden Layer ($l = 1$)

Let $W^{(1)}$ be the weight matrix connecting the input layer to the hidden layer and b^1 the bias vector. Each weight $w_{ij}^{(1)}$ connects input neuron j to hidden neuron i, and let $a_j^{(0)} = x_j$ be the input feature at position j.

The linear equation Z_i at the hidden neuron i is written as:

$$Z_i^{(1)} = \sum_j w_{ij}^{(1)} a_j^{(0)} + b_i^{(1)}.$$

Generally, it can be written as:

$$Z^{(1)} = W^{(1)} \cdot x + b^{(l)}.$$

Using the chain rule of differentiation, the gradient of loss becomes

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial Z^{(2)}} \frac{\partial Z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial Z^{(1)}} \frac{\partial Z^{(1)}}{\partial W^{(1)}}$$

and

$$\frac{\partial \mathcal{L}}{\partial b^{(1)}} = \frac{\partial \mathcal{L}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial Z^{(2)}} \frac{\partial Z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial Z^{(1)}} \frac{\partial Z^{(1)}}{\partial b^{(1)}}.$$

Recall that $\frac{\partial \mathcal{L}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial Z^{(2)}} = D^2$ is already computed at the output layer. Now to compute $\frac{\partial Z^{(2)}}{\partial a^{(1)}}$, you may recall that the output layer contains only one neuron and the linear equation at that layer is written as:

$$Z_1^{(2)} = W^{(2)} \cdot a^{(1)} = \sum_j W_{1j}^{(2)} a_j^{(1)} + b_1^2$$

Now,

$$\frac{\partial Z_1^{(2)}}{\partial a_k^{(1)}} = W_{1k}^{(2)}. \tag{7.28}$$

This can be written more generally as:

$$\frac{\partial Z_p^{(2)}}{\partial a_k^{(1)}} = W_{pk}^{(2)} \tag{7.29}$$

$$\frac{\partial Z^{(2)}}{\partial a^{(1)}} = W^{(2)}. \tag{7.30}$$

The derivative of the activation $a^{(i)}$ with respect to the input $Z^{(i)}$ at layer i is given by:

$$\frac{da^{(i)}}{dZ^{(i)}} = \phi'\left(Z^{(i)}\right) \tag{7.31}$$

So, this will result to a column vector each containing the value of $\phi'(Z_i^1)$. Lastly,

$$\frac{\partial Z^{(l)}}{\partial W^{(l)}} = a^{(l-1)}. \tag{7.32}$$

For the hidden layer 1,

$$\frac{\partial Z^{(1)}}{\partial W^{(1)}} = a^{(0)} = x.$$

Putting it all together,

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \left(D^2 \cdot (W^{(2)})^\top\right) \frac{\partial a^{(1)}}{\partial Z^{(1)}} \odot (a^0)^\top \tag{7.33}$$

$$= \left(D^2 \cdot (W^{(2)})^\top\right) \frac{\partial a^{(1)}}{\partial Z^{(1)}} \odot x^\top \tag{7.34}$$

and,

$$\frac{\partial \mathcal{L}}{\partial b^{(1)}} = \left(D^2 \cdot (W^{(2)})^\top\right) \frac{\partial a^{(1)}}{\partial Z^{(1)}}. \tag{7.35}$$

Gradients of specific weight or bias entries at layer 1

Gradients for specific weight or bias entries allows us to compute how each individual weight and bias in the first layer should be adjusted to reduce the network's loss:

$$\frac{\partial \mathcal{L}}{\partial W_{ik}^{(1)}} = \left(D_1^{(2)} \cdot W_{1i}^{(2)}\right) \frac{\partial a_i^{(1)}}{\partial Z_i^{(1)}} a_k^{(0)} \tag{7.36}$$

$$= \left(D_1^{(2)} \cdot W_{1k}^{(2)}\right) \phi'(Z_i^{(1)}) x_k \tag{7.37}$$

and,

$$\frac{\partial \mathcal{L}}{\partial b_i^{(1)}} = \left(D_1^{(2)} \cdot W_{1i}^{(2)}\right) \phi'(Z_i^{(1)}) \tag{7.38}$$

Here:

- $W_{ik}^{(1)}$ is the weight connecting the kth input x_k to the ith neuron in the first hidden layer.
- $\frac{\partial \mathcal{L}}{\partial W_{ik}^{(1)}}$ shows how this weight affects the total loss $\mathcal{L}$.
- $D_1^{(2)} \cdot W_{1i}^{(2)}$ represents how much the ith neuron in the first layer influences the output layer's error.
- $\phi'\left(Z_i^{(1)}\right)$ is the derivative of the activation function at neuron i in layer 1.
- x_k is the kth input value.

Back Propagation of Loss

Notice that we computed the error term $\frac{\partial \mathcal{L}}{\partial Z^1}$ at the hidden layer ($l = 1$) using the error term and weight matrix from the output layer ($l = 2$):

$$D^{(1)} = D^{(2)} \cdot (W^{(2)})^\top \tag{7.39}$$

This shows that, for any number of layers, the error at a given layer can be computed from the error and weights of the next layer. This process of sending the error backward through the network is called backpropagation.

$$D^{(l)} = D^{(l+1)} \cdot (W^{(l+1)})^\top \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \tag{7.40}$$

Computing the error contributions of a specific neuron

The error contribution by the k'th neuron at the l'th layer can be computed as

$$D_k^{(l)} = \left(\sum_p D_p^{(l+1)} W_{pk}^{(l+1)} \right) \phi_k' \tag{7.41}$$

Here,

- The term $\left(\sum_p D_p^{(l+1)} W_{pk}^{(l+1)} \right)$ is the backpropagation term. It represents the total contribution of the kth neuron in layer l to the errors in all neurons p in the next layer $(l+1)$, weighted by the corresponding weights $W_{pk}^{(l+1)}$.
- $\phi' = \frac{\partial a_k^{(l)}}{\partial z_k^{(l)}}$ is the derivative of the activation function at the k'th neuron of layer-l.

Generalized Rules for Gradient Computation

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot \frac{\partial a^{(l)}}{\partial z^{(l)}} a^{(l-1)} \tag{7.42}$$

$$\frac{\partial \mathcal{L}}{\partial b(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot \frac{\partial a^{(l)}}{\partial z^{(l)}} \tag{7.43}$$

Generalized Rules for Gradient Computation of Specific Entries

$$\frac{\partial \mathcal{L}}{\partial w_{ik}^{(l)}} = D_i^{(l)} a_k^{(l-1)} \tag{7.44}$$

$$\frac{\partial \mathcal{L}}{\partial b_{(l)}} = D^l \tag{7.45}$$

Generalized Update Rules for Weights and Bias

$$W_{ik}^{(l)} \leftarrow W_{ik}^{(l)} - \alpha D_i^{(l)} a_k^{(l-1)}, \tag{7.46}$$

and,

$$b_i^{(l)} \leftarrow b_i^{(l)} - \alpha D_i^{(l)} a_k^{(l-1)} \tag{7.47}$$

7.7.8 Simplified Example: Backpropagation in 2-layer Feed-forward Neural Network

Inputs:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = 1, \quad \alpha = 0.1 \tag{7.48}$$

Initial Weights and Biases:

$$W^{(1)} = \begin{bmatrix} 0.2 & -0.3 \\ -0.1 & 0.4 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0.1 \\ 0.05 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 0.3 & -0.2 \end{bmatrix}, \quad b^{(2)} = 0.2$$

Step 1: Forward Pass

Hidden layer:

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{bmatrix} 0.2 \cdot 1 + (-0.3) \cdot 2 + 0.1 \\ -0.1 \cdot 1 + 0.4 \cdot 2 + 0.05 \end{bmatrix} = \begin{bmatrix} -0.3 \\ 0.75 \end{bmatrix} \tag{7.49}$$

$$a^{(1)} = \sigma(z^{(1)}) = \begin{bmatrix} \frac{1}{1+e^{0.3}} \\ \frac{1}{1+e^{-0.75}} \end{bmatrix} \approx \begin{bmatrix} 0.4256 \\ 0.6792 \end{bmatrix} \tag{7.50}$$

Step 2: Output Layer Backward Pass

$$\frac{d\mathcal{L}}{da^{(2)}} = \hat{y} - y = 0.5319 - 1 = -0.4681 \tag{7.51}$$

$$\frac{da^{(2)}}{dZ^{(2)}} = \hat{y}(1 - \hat{y}) \approx 0.5319(1 - 0.5319) \approx 0.2490 \tag{7.52}$$

$$D^{(2)} = \frac{d\mathcal{L}}{dZ^{(2)}} = -0.4681 \cdot 0.2490 \approx -0.1166 \tag{7.53}$$

Step 3: Backpropagate to Hidden Neurons

Hidden Layer 1:

Error term for neuron 1:

$$D_1^{(1)} = D_1^{(2)} \cdot w_{11}^{(2)} \cdot a_1^{(1)}(1 - a_1^{(1)}) = -0.1166 \cdot 0.3 \cdot 0.4256(1 - 0.4256) \approx -0.0085 \tag{7.54}$$

Error term for neuron 2:

$$D_2^{(1)} = D_1^{(2)} \cdot w_{12}^{(2)} \cdot a_2^{(1)}(1 - a_2^{(1)}) = -0.1166 \cdot (-0.2) \cdot 0.6792(1 - 0.6792) \approx 0.0051 \tag{7.55}$$

Step 4: Update Hidden Layer Weights

Hidden Layer 1:

Neuron 1:

$$w_{11}^{(1)} \leftarrow w_{11} - \alpha D_1^{(1)} x_1 \tag{7.56}$$
$$= 0.2 - 0.1 \cdot (-0.0085) \cdot 1 \tag{7.57}$$
$$= 0.20085 \tag{7.58}$$

$$w_{12}^{(1)} \leftarrow w_{12} - \alpha D_1^{(1)} x_2 \tag{7.59}$$
$$= -0.3 - 0.1 \cdot (-0.0085) \cdot 2 \tag{7.60}$$
$$= -0.2983 \tag{7.61}$$

$$b_1^{(1)} \leftarrow b_1^1 - \alpha D_1^{(1)} \tag{7.62}$$
$$= 0.1 - 0.1 \cdot (-0.0085) \tag{7.63}$$
$$= 0.10085 \tag{7.64}$$

Neuron 2:

$$w_{21}^{(1)} \leftarrow w_{21} - \alpha D_2^{(1)} x_1 \tag{7.65}$$
$$= -0.1 - 0.1 \cdot 0.0051 \cdot 1 = -0.10051 \tag{7.66}$$
$$w_{22}^{(1)} \leftarrow w_{22} - \alpha D_2^{(1)} x_2 \tag{7.67}$$
$$= 0.4 - 0.1 \cdot 0.0051 \cdot 2 = 0.39898 \tag{7.68}$$
$$b_2^{(1)} \leftarrow b_2 - \alpha D_2^{(1)} \tag{7.69}$$
$$= 0.05 - 0.1 \cdot 0.0051 = 0.04949 \tag{7.70}$$

7.7.9 Example: Backpropagation on a 3-Layer Neural Network

Network Structure

- Input Layer: $x_1 = 1.0,\ x_2 = 2.0$
- Hidden Layer 1 (3 neurons): sigmoid activation
- Hidden Layer 2 (2 neurons): sigmoid activation
- Output Layer (1 neuron): sigmoid activation

Randomly Initialized Weights and Biases

Layer 1 (Input to Hidden 1):

$$W^{(1)} = \begin{bmatrix} 0.1 & -0.2 \\ 0.4 & 0.3 \\ -0.5 & 0.2 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0.0 \\ 0.1 \\ -0.1 \end{bmatrix}$$

Layer 2 (Hidden 1 to Hidden 2):

$$W^{(2)} = \begin{bmatrix} 0.2 & -0.1 & 0.3 \\ -0.4 & 0.2 & 0.1 \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

Layer 3 (Hidden 2 to Output):

$$W^{(3)} = \begin{bmatrix} 0.3 & -0.2 \end{bmatrix}, \quad b^{(3)} = 0.1$$

Step 1: Forward Pass

Compute pre-activations and activations:

$$z^{(1)} = W^{(1)} \cdot x + b^{(1)} = \begin{bmatrix} 0.1(1) + (-0.2)(2) \\ 0.4(1) + 0.3(2) \\ -0.5(1) + 0.2(2) \end{bmatrix} + \begin{bmatrix} 0 \\ 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} -0.3 \\ 1.1 \\ -0.2 \end{bmatrix}$$

$$a^{(1)} = \sigma(z^{(1)}) = \begin{bmatrix} \sigma(-0.3) \\ \sigma(1.1) \\ \sigma(-0.2) \end{bmatrix} = \begin{bmatrix} 0.4256 \\ 0.7503 \\ 0.4502 \end{bmatrix}$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = \begin{bmatrix} 0.2 & -0.1 & 0.3 \\ -0.4 & 0.2 & 0.1 \end{bmatrix} \begin{bmatrix} 0.4256 \\ 0.7503 \\ 0.4502 \end{bmatrix} + \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix} = \begin{bmatrix} 0.1734 \\ -0.1800 \end{bmatrix}$$

$$a^{(2)} = \sigma(z^{(2)}) = \begin{bmatrix} 0.5432 \\ 0.4551 \end{bmatrix}$$

$$z^{(3)} = W^{(3)}a^{(2)} + b^{(3)} = [0.3, -0.2] \begin{bmatrix} 0.5432 \\ 0.4551 \end{bmatrix} + 0.1 = 0.1629$$

$$\hat{y} = a^{(3)} = \sigma(0.1629) = 0.5406$$

Step 2: Backward Pass

Output layer delta:

$$D^{(3)} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \cdot \sigma'(z^{(3)}) = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) = (0.5406 - 1)(0.5406)(0.4594) = -0.1139$$

Layer 2 delta using:

$$D_k^{(2)} = \left(\sum_p D_p^{(3)} W_{pk}^{(3)} \right) \cdot \sigma'(z_k^{(2)})$$

$$\text{Since } D^{(3)} = -0.1139, \quad W^{(3)} = [0.3, -0.2]$$

$$D_1^{(2)} = (-0.1139)(0.3) \cdot \sigma'(0.1734) = -0.03417 \cdot (0.5432)(0.4568) = -0.0085$$

$$D_2^{(2)} = (-0.1139)(-0.2) \cdot \sigma'(-0.1800) = 0.02278 \cdot (0.4551)(0.5449) = 0.0057$$

Layer 1 delta:

$$D_k^{(1)} = \left(\sum_p D_p^{(2)} W_{pk}^{(2)} \right) \cdot \sigma'(z_k^{(1)})$$

$$\begin{aligned} D_1^{(1)} &= (-0.0085 \cdot 0.2 + 0.0057 \cdot (-0.4)) \cdot \sigma'(-0.3) \\ &= (-0.0017 - 0.00228) \cdot 0.2445 \\ &= -0.00097 \end{aligned}$$

$$\begin{aligned} D_2^{(1)} &= (-0.0085 \cdot (-0.1) + 0.0057 \cdot 0.2) \cdot \sigma'(1.1) \\ &= (0.00085 + 0.00114) \cdot 0.1879 \\ &= 0.00037 \end{aligned}$$

$$\begin{aligned} D_3^{(1)} &= (-0.0085 \cdot 0.3 + 0.0057 \cdot 0.1) \cdot \sigma'(-0.2) \\ &= (-0.00255 + 0.00057) \cdot 0.2475 \\ &= -0.00049 \end{aligned}$$

Step 3: Gradient Update for Weights and Biases

Using:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(l)}} = -\alpha D_i^{(l)} \cdot a_j^{(l-1)} \qquad \frac{\partial \mathcal{L}}{\partial b_i^{(l)}} = -\alpha D_i^{(l)}$$

Assume learning rate $\alpha = 0.1$, then for example:

$$\frac{\partial \mathcal{L}}{\partial W_{1,1}^{(1)}} = -0.1 \cdot (-0.00097) \cdot 1.0 = 0.000097$$

$$\frac{\partial \mathcal{L}}{\partial W_{1,2}^{(1)}} = -0.1 \cdot (-0.00097) \cdot 2.0 = 0.000194$$

$$\frac{\partial \mathcal{L}}{\partial b_1^{(1)}} = -0.1 \cdot (-0.00097) = 0.000097$$

Similarly, compute updates for all weights and biases.

What Happens After We Update the Weights?

Once the weights and biases are updated using backpropagation, they are used in the next iteration of training. Here's what happens:

- In the **next forward pass**, the updated weights and biases are used to make a new prediction.

- If the prediction is still incorrect, **new gradients are computed** based on the updated prediction, and weights are further updated.
- This cycle repeats over many training examples and epochs until the network **converges to a solution** that minimizes the loss.
- Once training is complete, the final set of weights is **frozen and used for inference** on unseen data.

Backpropagation trains a neural network by making small improvements to its weights with each example it sees. These updates accumulate over time, gradually improving the network's predictions.

Training a Neural Network: Step-by-Step Procedure

Training a neural network involves iteratively adjusting its parameters to minimize prediction error on a dataset. The process uses feedforward computation, loss evaluation, backpropagation, and weight updates. Below are the key steps:

Step 1: Initialize Parameters

- Randomly initialize the weights $W^{(l)}$ and biases $b^{(l)}$ for each layer l.
- Choose a learning rate η, number of epochs, and batch size (if applicable).

Step 2: Forward Pass

- For each input x, compute the outputs of each layer using:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)})$$

- The final activation $a^{(L)}$ is the predicted output $\hat{y}$.

Step 3: Compute Loss

- Compare the predicted output $\hat{y}$ with the true label y using a suitable loss function, e.g., binary cross-entropy for classification:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Step 4: Backpropagation

- Compute gradients of the loss w.r.t. output layer and recursively propagate errors backward:

$$\delta^{(L)} = (\hat{y} - y) \odot \sigma'(z^{(L)})$$
$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(z^{(l)})$$

- Calculate gradients of weights and biases:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

Step 5: Gradient Descent at Each Layer

- Use the computed gradients to update the weights and biases layer by layer:

$$\text{For each layer } l = 1, 2, \ldots, L:$$

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

- This ensures that each layer's parameters are adjusted to reduce the error based on their contribution.

Step 6: Repeat for All Examples and Epochs

- Repeat steps 2 to 5 for each training example or mini-batch.
- After each pass through the entire dataset, increment the epoch counter.
- Continue until the loss converges or a stopping condition is met.